

ClipMind AI: Video Summarization & KeyMoments Detection

Milestone 1: Project Initialization, Design Process and Core Setup (Module 1)

1. Team Details

- **Project Title:** ClipMind AI: Video Summarization & KeyMoments Detection
- **Batch Number:** 3
- **Team:** 4
- **Start Date:** 19 August 2026
- **Internship Duration:** 8 weeks

My Work Responsibility

- **Module 1:** Backend setup and core APIs: FastAPI setup, registration, login, JWT authentication, role-based access control, video upload, validation and storage APIs, upload history and status APIs, and the MongoDB Atlas database integration.
- **Module 2:** Transcript and Summary APIs that connect the Whisper and AI summarization pipeline to the database and the frontend.

2. Project Name

ClipMind AI: Video Summarization & KeyMoments Detection: an AI-powered platform that converts lecture and video content into transcripts, summaries and key moments.

3. Project Objective

The objective of Milestone 1 is to establish the complete foundation of the ClipMind AI application so that all upcoming modules can be built on a stable base. By the end of this milestone, a user should be able to register, log in securely, access the application according to their role, upload a video, and view upload history and processing status.

My objective within this milestone was to design, build and verify the backend layer: the FastAPI service, secure authentication, role-based access control, video upload APIs and the database integration. In Module 2, my objective was to provide the Transcript and Summary APIs that deliver the AI pipeline's results to the frontend.

4. Project Description and Main Features

ClipMind AI follows a layered architecture:

User, then Frontend (React/Next.js), then FastAPI Backend, then Authentication and Role Management, then Database (MongoDB Atlas), then Video Upload and Storage, then FFmpeg Processing.

AI features such as transcription, summarization, key moments and analytics belong to later modules and are not part of this milestone's implemented flow.

Module 1 Features: Backend APIs and Database

1. User Registration

- Creates a new user account with name, email, password and role.
- Checks for duplicate email addresses.
- Enforces a minimum password length of 8 characters.
- Restricts the role to four valid values: content_creator, learner, educator, administrator.
- Stores passwords as bcrypt hashes, never as plain text.

2. User Login and JWT Authentication

- Verifies credentials and returns a signed JWT access token.
- Protected APIs validate the token before granting access.
- Expired tokens return a distinct message ("Your session has expired. Please log in again.") from invalid tokens ("Could not validate credentials").

3. Role-Based Access Control (RBAC)

- A reusable require_role() dependency controls access by role.
- One example protected route exists for each of the four roles.
- Users with the wrong role receive 403 Forbidden.

4. Video Upload, Validation and Storage

- Authenticated users can upload videos.
- Validation covers file type, empty (0-byte) files, and files larger than 500 MB.
- Files are stored locally and metadata is saved in MongoDB, linked to the uploading user.

5. Upload History and Processing Status

- History endpoint returns the logged-in user's uploaded videos.
- Status endpoint returns the processing status of a single video.

6. Database Migration to MongoDB Atlas

- The backend was migrated from SQLite (temporary placeholder) to MongoDB Atlas after the team decided on MongoDB.
- Implemented with the Motor async driver and the Beanie ODM, with all endpoints rewritten as asynchronous.

Module 2 Features: Transcript and Summary APIs

In Module 2, I developed the backend Transcript and Summary APIs, which connect the Whisper and AI summarization pipeline to the database and the frontend. The pipeline flow is: Video Upload, Audio Extraction (FFmpeg), Whisper, Transcript, AI Summarization, Short and Detailed Summary, Display to User. The APIs implement these Module 2 requirements:

- **Transcript API:** returns the transcript of a video; the transcript is stored once in MongoDB and not regenerated on every page visit.
- **Summary API:** returns the short and detailed summaries and supports generating or regenerating a summary.
- **Access control:** each transcript and summary belongs to one user and one video, and only authorized users can read it.
- **Status tracking:** NOT_STARTED, PROCESSING, COMPLETED and FAILED statuses are returned so the frontend can show progress.
- **Duplicate-processing guard:** no new summarization job starts while one is already PROCESSING.
- **Failure handling:** the transcript is kept if summarization fails, and the user can retry.

Module 2 Requirements

Module 2 (Transcript Generation and AI Summarization) takes an uploaded lecture video, converts its speech into a text transcript, and uses AI to produce a summary. The requirements are:

1. **Transcript generation:** extract audio from the video with FFmpeg and convert it to text with Whisper.
2. **Transcript storage:** store the transcript in the database once, linked to the correct user and video, so it is not regenerated on every visit.
3. **Transcript page:** show the video title, upload date, transcript status and transcript text, with a loading indication while processing.
4. **AI summarization:** pass the transcript to a Hugging Face model (BART or T5) to produce a concise summary.
5. **Long transcript handling:** split very long transcripts into chunks, summarize each chunk, and combine them into a final summary.

6. **Short and detailed summaries:** provide a short summary of the main idea and a detailed summary covering the major concepts.
7. **Summary page:** show both summaries separately, with status, last generated time, a generate/regenerate option, and an error message on failure.
8. **Status tracking:** use NOT_STARTED, PROCESSING, COMPLETED and FAILED so the user always sees the current state.
9. **Duplicate prevention:** do not start a new summarization job while one is already PROCESSING.
10. **Failure handling:** keep the transcript if summarization fails, and let the user retry.

Module 2 API Build

The Transcript and Summary APIs were built on the same FastAPI backend as Module 1, so they reuse its JWT authentication and role checks. The build followed these steps:

1. **Data models:** added Transcript and Summary documents in MongoDB (with Beanie), each linked to a user and a video and holding the text, status and timestamps.
2. **Routers:** added the Transcript and Summary routers to the FastAPI app, registered alongside the auth, users and videos routers.
3. **Route protection:** every route requires a valid JWT, and a user can read only the transcripts and summaries of their own videos.
4. **Transcript API:** returns the stored transcript and its status; the Whisper output is saved once and reused.
5. **Summary API:** returns the short and detailed summaries, and checks the current status before starting a job so that duplicate processing is avoided.
6. **Failure handling:** a failed summarization sets the status to FAILED, keeps the transcript, and allows a retry.
7. **Testing:** the endpoints were tested through Swagger UI and Postman.

API Endpoints

- **POST /api/auth/register** (no authentication): creates a new user account.
- **POST /api/auth/login** (no authentication): verifies credentials and returns a JWT.
- **GET /api/users/me** (authenticated): returns the logged-in user's profile.
- **GET role-specific example routes, one per role** (authenticated): demonstrate role-based access.
- **POST /api/videos/upload** (authenticated): uploads and validates a video.
- **GET /api/videos/history** (authenticated): lists the user's uploaded videos.
- **GET /api/videos/{id}/status** (authenticated): returns a video's processing status.

Backend Project Structure

The backend is organized into `app/` with the sub-packages `core` (config, security, roles), `db` (database connection), `models` (User, Video), `schemas` (request and response shapes), and `api` (dependencies and routes for auth, users and videos), along with `requirements.txt` and `.env.example`.

Version Control

The backend was pushed to the shared GitHub repository (springboardmentor4132x/AI--Video-Summarizer-4) on the personal branch `tejashwinigm-AI--Video-Summarizer-4`. The `.env` file, virtual environment and cache files were excluded through `.gitignore`.

5. Technology Used

- **Language:** Python 3.13.7
- **Backend Framework:** FastAPI with Uvicorn
- **Database:** MongoDB Atlas (M0 cloud cluster)
- **Database Driver / ODM:** Motor (async driver) and Beanie
- **Authentication:** JWT (python-jose)
- **Password Security:** bcrypt
- **Validation:** Pydantic, email-validator
- **File Uploads:** python-multipart
- **Configuration:** python-dotenv, pydantic-settings
- **AI Pipeline (Module 2 integration):** FFmpeg, Whisper, Hugging Face Transformers (BART/T5)
- **API Testing:** Swagger UI (`/docs`) and Postman
- **Version Control:** Git and GitHub
- **Development Tool:** Visual Studio Code

6. Screenshots

Insert the screenshots below before final submission.

Figure 1: FastAPI server running (terminal showing "Application startup complete")

```
INFO:      Stopping Reloader process [9632]
(venv) PS C:\Users\HP\OneDrive\Desktop\ClipMind-AI Video Summarizer\clipmind\backend> python -m uvicorn a
pp.main:app --reload
INFO:      Will watch for changes in these directories: ['C:\\Users\\HP\\OneDrive\\Desktop\\ClipMind-AI Vi
deo Summarizer\\clipmind\\backend']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [4444] using WatchFiles
INFO:      Started server process [7856]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Figure 2: Swagger UI (<http://127.0.0.1:8000/docs> showing Auth, Users and Videos routers)

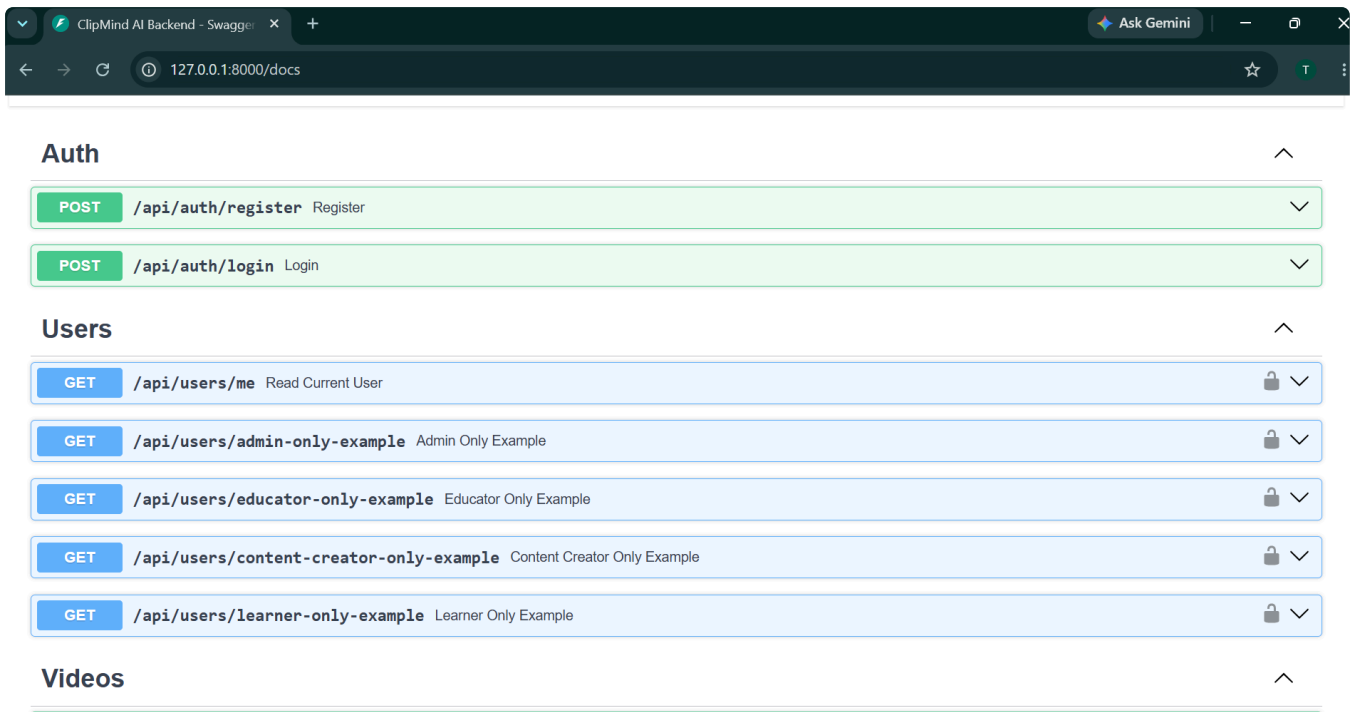


Figure 3: Successful registration (HTTP 201 response)

Request URL	
<code>http://127.0.0.1:8000/api/auth/register</code>	
Server response	
Code	Details
201	Response body <pre>{ "id": "6ac7f21db0b4ec6b09e0c257", "name": "Teju", "email": "use@example.com", "role": "learner", "created_at": "2026-10-08T19:42:21.846050Z" }</pre>

Figure 4: Successful login (HTTP 200 with JWT access token)

Request URL	
<code>http://127.0.0.1:8000/api/auth/login</code>	
Server response	
Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3ZDIwIjIyTkZDhhNzRjMmIwYzdjNjM1MDQwOGM1LCJyb2x1IjoibGVhcm5lc1IsImV4cCI6MTc5MTQ5MjI0MH0.ngZwZwefs_vqhGxspHdGDSBa-brYwxLSb609jDNHvro", "token_type": "bearer" }</pre>  

7. Challenges Faced

- **Python not recognized.** Windows redirected the python command to a Microsoft Store prompt. I installed Python 3.13.7 from python.org with "Add python.exe to PATH" enabled.
- **Project folder not visible on the Desktop.** The folder was created in a different path from the OneDrive-synced Desktop. I moved it to the OneDrive Desktop path.
- **Server crash on first run (ImportError for auth).** The auth.py file had not been saved. I checked the files on disk, recreated auth.py, and the server started cleanly.
- **Database choice undecided.** The plan required a team decision between PostgreSQL and MongoDB. I used SQLite temporarily and migrated to MongoDB Atlas once the team decided.

- **Rewriting the database layer.** SQL tables and MongoDB documents work differently. I rewrote the models, schemas, dependencies and routes using Motor and Beanie with async queries.
- **Gaps against plan requirements.** The first build lacked password strength checks, expired-token handling, RBAC for all four roles and file-size validation. I ran a gap analysis, added each missing item and verified it with live tests.
- **Git repository not initialized.** The local backend folder was not a Git repository. I cloned the shared repo, copied the source, added a .gitignore and pushed to my personal branch.
- **Pull Request could not be opened.** The shared repo had no main branch and the branches had unrelated histories. I raised it for team coordination; the PR is pending a main branch.
- **Duplicate file.** Both `app/api/auth.py` and `app/api/routes/auth.py` exist. I flagged it for cleanup before merge.

8. Conclusion

Milestone 1 established the working foundation of ClipMind AI. On the backend, a FastAPI service now supports secure registration, login with JWT, role-based access for all four user roles, validated video upload, upload history and status tracking, all backed by MongoDB Atlas. Each feature was verified with live API requests.

The backend code is version-controlled on a personal GitHub branch. Remaining items for the team are creating a main branch so Pull Requests can be opened, removing the duplicate `auth.py` file, tightening CORS and Atlas network access before deployment, and integrating the backend with the frontend and the FFmpeg processing pipeline.

In Module 2, I developed the Transcript and Summary APIs, which carry the output of the Whisper and AI summarization pipeline to the database and the frontend, with status tracking, protection against duplicate processing and failure handling.